REPLACING REPOSITORY EXPLORATION WITH TYPED JEV DECISIONS
CS329Z MIDWAY REPORT

Kristjan Dagur Egilsson **Ómar Ingi Halldórsson** **Rami Ratl Mrad**

1 RESEARCH QUESTIONS

Our project asks whether a structured decision model can replace a narrow component of an existing coding agent at lower total cost while preserving bug-repair performance. Following the instructor's recommendation to obtain mechanical evidence before redesigning an agent harness, we study OpenCode's repository-exploration subagent. Our intervention uses Jev to select relevant source files from deterministic search candidates; the same strong generative model continues to write and test patches. We compare this intervention against the native generative explorer and a search-only replacement. The primary outcomes are independently graded repair success, total inference cost, and wall time.

We report four experiments. A development pilot on five SWE-bench Lite issues fixed the method. A second pilot applied the unchanged method to ten SWE-bench Verified tasks, graded by the official containerized harness. An extended development run covers 41 further pydicom issues with an oracle upper bound and per-phase timing. Finally, an offline study measures Jev's file selection directly on all 225 SWE-bench development issues. The midpoint objective is a working substitution and an authentic evaluation pipeline, not a claim of calibrated confidence or performance equivalence from small samples.

2 ENVIRONMENT & DATA

Existing agent harness. We use the pinned OpenCode 1.19.5 command-line harness and retain its native build agent, `task` delegation tool, `explore` subagent, file tools, and patch-writing loop (OpenCode contributors, 2026). Each rollout starts from an exact upstream repository snapshot without upstream git history. The issue prompt asks the build agent to delegate initial localization to the existing explorer before editing, then implement a minimal fix and optionally run existing tests. The prompt is identical across conditions within an experiment. The parent agent retains its own read and search tools and can therefore recover from imperfect localization. Tool permissions allow edits only to the repository's implementation directories, never to tests; these permissions are not an operating-system sandbox.

Development pilot (SWE-bench Lite). We used all five pydicom issues in the official SWE-bench Lite development split (Jimenez et al., 2024; SWE-bench contributors, 2026a), chosen for a locally runnable pure-Python environment. Task IDs and base commits were frozen before agent outcomes. Each patch was evaluated by applying it and the benchmark's test patch to a fresh snapshot and running the published fail-to-pass and pass-to-pass tests under pinned local pytest. Every environment was validated beforehand: the broken base fails and the upstream fix passes. This local evaluation is not an official SWE-bench score.

Extended development run (41 pydicom issues). A second development protocol, frozen before any of its runs, takes every remaining pydicom issue in the full SWE-bench development split (Jimenez et al., 2024): 51 issues, of which 41 pass the same environment controls and 10 are dropped and listed because even their upstream fix fails in the pinned environment (removed NumPy APIs, Python 3.12 behaviour, uncollectable tests). One amendment, recorded before any run, accepts a broken base whose selected tests error rather than fail, for five issues whose test patch imports an API the fix adds. Evaluation is the local pytest procedure above.

Verified pilot sample. SWE-bench Verified is a 500-task subset of the SWE-bench test split whose issues and tests were screened by human annotators for adequate specification (OpenAI, 2024). We pinned dataset revision `78f471b` (SWE-bench contributors, 2026b) and drew a 50-task sample, stratified by repository in proportion to the full set; within each repository, tasks are ordered by a SHA-256 hash of a recorded seed and the instance ID. The first pilot uses a nested 10-task subset produced by the same rule: five Django tasks, two SymPy, and one each from Matplotlib, scikit-learn, and Sphinx. Selection used only instance IDs and repository names. Both the sample and the protocol were committed before any agent run.

Official evaluation and environment checks. Verified patches are graded by the official `swebench` 5.0.2 harness (Jimenez et al., 2024) using the published per-task Docker images and a local copy of the pinned rows. Before any paid run, we checked every image. Each contains the base commit plus one harness setup commit and no later upstream history. That setup commit changes no source or test content; it only alters file permissions, except for dependency pins in Sphinx's packaging files. Neither the benchmark's test patch nor the gold fix is present in any image, and all ten gold fixes resolve under the harness. Public issues may appear in model training data, so we make no contamination-free claim.

File-selection evaluation on 225 development issues. End-to-end success cannot isolate the one decision Jev makes, and the parent agent can recover from a poor file choice. We therefore also evaluate that decision directly on every instance of the full SWE-bench development split (Jimenez et al., 2024): 225 real issues from six Python repositories (pydicom, sqlfluff, pvlib, astroid, pyvista, marshmallow). Each instance's *gold files* are the implementation files its official fix modifies; they are used only by the scorer. For each issue we build the same 12 candidates as the agent, from the issue text at the base commit, and compare the three files chosen by search alone and by Jev. The primary metric is *hit@3*, the share of issues with at least one gold file among the three chosen. We also report candidate recall (a gold file among the 12, the ceiling for any selector) and hit@3 restricted to issues where retrieval found a gold file. Jev and search are compared on the same issues with a paired bootstrap interval and an exact sign test. The protocol was committed before any decision was requested. The SWE-bench test split, apart from the Verified pilot sample, is reserved for held-out final evaluation.

3 METHODS

One internal substitution. The build agent uses `openai/gpt-6.1-sol` in every condition. A thin OpenAI-compatible provider adapter exposes distinct build and explore URLs. OpenCode's native `task` tool still creates an explorer session and returns its result to the parent. In the native condition, both URLs forward to the same strong model, so the explorer is a GPT-6.1-Sol subagent with read-only tools. In the Jev condition, only explore-provider requests are replaced by deterministic retrieval, one typed Jev request, and a formatted list of selected file paths and snippets.

Candidate construction and Jev decisions. We index the repository's Python implementation files, excluding tests. A deterministic BM25-style search ranks files against the parent agent's actual exploration request. The top 12 candidates include relative paths and the most query-matching 16-line window of each file, capped at 1,800 characters. Jev 1.14 receives the request and candidates and answers one Noul file-relevance question per candidate in a single API call (OpenRouter, 2026). We validate every returned probability and return the three highest-scoring files to the parent. Jev neither writes patches nor selects the coding model. An API error or invalid score falls back to the native explorer, and both costs are counted. No confidence threshold is applied, and relevance scores are not calibrated probabilities of repair success. The offline file-selection study reuses this retrieval and the identical Jev question, generalized from `pydicom/` to each repository's package directory; it reproduces the development pilot's ten archived candidate lists exactly. Its one difference is the query: the issue text, rather than the parent agent's exploration request.

Comparisons and fixed limits. Each experiment runs three conditions on the same tasks, one rollout each: native exploration, retrieval plus Jev selection, and search-only, which returns the top three retrieval candidates without Jev. The search-only control tests whether any benefit requires the decision model. The extended development run adds an *oracle* condition whose explorer returns

Table 1: Both pilots, one rollout per task and condition; cost includes all captured model calls; time is agent wall time excluding grading. Top: five pydicom Lite development issues, local regression tests. Bottom: ten SWE-bench Verified tasks, official Docker harness, with mean cost split into the explore step and the parent writer.

Condition	Solved	Cost (USD)	Explore	Writer	Time (s)
Development pilot (pydicom, SWE-bench Lite dev)					
Native generative explorer	2/5	0.110			37.6
Jev file selection	2/5	0.073			25.9
Search only	2/5	0.062			24.9
Verified pilot (10 tasks, 5 repositories)					
Native generative explorer	9/10	0.106	0.0572	0.049	42.9
Jev file selection	9/10	0.056	0.0002	0.056	26.0
Search only	8/10	0.054	0.0000	0.054	25.3

the implementation files changed by the upstream fix (paths and the usual snippets only, never the fix itself). It is an upper bound on what better localization can buy, not a deployable method. All conditions share the writer model, tools, evaluator, 16 parent steps, 6 explorer steps, at most 32 requests, 4,096 output tokens per request, and 480 seconds per run. The Verified protocol imports these settings from the development protocol and refuses to run if any of them differ.

Agent-side testing on Verified. Verified repositories require version-specific environments, so the agent's test command runs inside the task's official image. Each run starts one container, completes the image's environment setup online, then disconnects the network and replaces the repository's git history with a single fresh commit. The agent may call `run_tests` with test targets. The wrapper applies the current workspace diff to a clean copy and invokes the repository's standard test runner with the validated, shell-quoted targets. The benchmark's own test targets, test patch, and gold patch never reach the agent.

Accounting, timing, and artifacts. The adapter records upstream responses and billed usage for writer, auxiliary, exploration, and Jev calls. A durable ledger reserves conservative costs before requests and settles bills; caps are $2 per rollout, $10 for the Verified pilot, and $25 for the extended run. The real API key stays in the host adapter. The extended run also records each run's time split: exploration (retrieval plus the Jev decision, or the native explorer's model calls), writer model calls, and grading. Each run stores its source/protocol hash, task metadata, event trace, request/response receipts, file decisions, test-command log, patch, and grader report. All 30 Verified pilot runs share one source hash, and every bill was captured.

4 Summary of Progress

Measured comparison. Table 1 summarizes both pilots. On the five development issues, the native and Jev conditions each solved 2/5, with mean cost falling from $0.110382 to $0.072762 (34.1%). On the ten Verified tasks, native and Jev each resolved 9/10 and search-only resolved 8/10. Mean total cost fell from $0.106 for native to $0.056 for Jev, a 48% reduction, and mean agent time fell from 42.9 to 26.0 seconds. The Jev runs used ten decision calls and no fallbacks. They replaced 48 explorer generation calls, which cost the native condition $0.057 and 21.7 seconds per task on average. There were no timeouts or forbidden edits. The Verified pilot cost $2.17 OpenRouter.

Extended run: time and cost fall, success does not change. On the 41 further pydicom issues (Table 2), Jev changed mean agent time by -16.3 s per issue relative to the native explorer (-35%; 95% paired bootstrap interval -19.1 to -13.7 s) and mean cost by -35%. Nearly all of the saving is exploration: 21.6 s of model calls for the native explorer against 1.6 s for retrieval and one Jev decision, while writer time is similar in every condition. Repair success did not differ: 18, 19, 21 and 20 of 41 issues for native, Jev, search and oracle, with at most three discordant issues in any pairwise comparison. Even the oracle, which is handed the files the upstream fix changes, repaired no more issues; on these issues localization is not the limiting step, writing the correct change is.

Table 2: Extended development run: 41 pydicom issues, one rollout per issue and condition, local regression tests. Time is mean agent wall time; explore is its exploration part. Last column: the explorer's report named a file changed by the upstream fix.

Condition	Solved	Cost (USD)	Time (s)	Explore (s)	Gold file named (%)
Native generative explorer	18/41	0.1390	46.5	21.6	98
Jev file selection	19/41	0.0910	30.2	1.6	93
Search only	21/41	0.0868	28.7	1.4	63
Oracle (gold files)	20/41	0.0908	27.4	0.0	100

Table 3: File selection on the SWE-bench development split (one Jev decision per issue). Recall@12: a gold file is among the 12 candidates. Hit@3: a gold file is among the 3 chosen. W/L: issues where only Jev, or only search, chose a gold file.

Repository	Issues	Recall@12 (%)	Search hit@3 (%)	Jev hit@3 (%)	W/L
marshmallow	9	100.0	100.0	100.0	0/0
pvlib	63	95.2	88.9	93.7	4/1
pydicom	56	92.9	67.9	85.7	11/1
astroid	31	90.3	61.3	87.1	8/0
pyvista	16	75.0	43.8	68.8	5/1
sqlfluff	50	78.0	58.0	76.0	10/1
All	225	88.9	70.2	85.3	38/4

Jev and search did not differ on time (1.4 s, interval -0.2 to 3.1), cost (5%) or success, although Jev named a gold file far more often. A final patch that edited no gold file never passed (0 of 8 runs), but the parent located the gold file itself in 95% of all runs, including those whose explorer report missed it. The extended run cost $16.71.

Jev chooses files better than search. On all 225 development issues (Table 3), Jev placed a gold file among its three choices for 85.3% of issues, compared with 70.2% for search alone and 30.8% expected from a random choice among the same candidates. The paired difference is 15.1 percentage points (95% bootstrap interval 9.8 to 20.4). Jev was right where search was wrong on 38 issues and the reverse on 4 (sign test $p < 0.001$). When retrieval found a gold file (200 issues), Jev chose one 96.0% of the time, versus 79.0% for search; the remaining errors are mostly retrieval misses, so candidate recall (88.9%) bounds the selector. The gain holds in every repository except marshmallow, where both are perfect. The study cost $0.0491. In the 15 development pilot rollouts the explorer's report contained a gold file in 5/5 native, 3/5 Jev and 3/5 search runs, yet all 15 final patches edited the gold file.

Longer shortlists. A separately frozen sweep repeated the study with 24 and 48 candidates. Jev's hit@3 rose from 85.3% to 90.2% and 91.1%: of the 25 issues whose gold file missed the 12 candidates, Jev recovered 13 at 24 and 15 at 48 (two are files created by the fix), while keeping 189 of the 192 issues it already solved at 12. The 24-candidate gain over 12 is 4.9 points (95% interval 1.3 to 8.4; 14 wins, 3 losses, $p = 0.013$). Median decision latency grew only from 0.29 to 0.34 s, but at 48 candidates 44 of 225 decisions took about 8 s, against 0 at 24; decisions cost $0.22, $0.39 and $0.69 per thousand. A 24-candidate shortlist therefore captures most of the benefit without the latency tail.

Where the saving comes from. The search-only control matched Jev's cost and came within one task of its success, so the experiment does not show that Jev is preferable to deterministic retrieval. Its single additional failure (Django 15695) is not a retrieval failure: search returned the file containing the upstream fix, and the writer edited it but placed its guard after an index lookup that fails. To examine localization separately from success, we compared the returned files with the files changed by the upstream fix. This analysis uses evaluator-side data that never reached the agent. The fix file appeared among the three returned files in 7/10 Jev runs and 6/10 search-only runs. Nonetheless, the writer edited that file in all 30 Verified runs. In the traces we inspected where the returned files missed the fix file, the parent issued its own searches immediately after delegating exploration and located the file itself. The saving therefore comes primarily from not

running a generative exploration subagent whose work the parent largely repeats, rather than from the quality of Jev's selections. Jev's highest relevance score also did not separate correct from incorrect selections in this sample: misses scored up to 0.85, and hits as low as 0.66. All three conditions failed SymPy 13852, and the development pilot's failures likewise reflected incomplete repairs, not infrastructure errors. The extended run and the file-selection study agree: Jev selects files better than search, but a strong parent with free search tools recovers from poor selections, on large repositories such as Django as well as on pydicom, so better selection does not yet translate into repairs, time, or cost.

Completed work and next steps. We have completed the development pilot, a Linux-validated reproduction, the Verified pipeline with environment checks, the 30-run Verified pilot, the 164-run extended development run, and the offline file-selection study and shortlist sweep. The Verified pipeline has 27 offline checks covering method equivalence with the development protocol, leakage boundaries, test-command validation, and grading edge cases; the shared development runner keeps its 16 offline checks. The file-selection studies and the extended run can be recomputed offline from their archives. Next, we will test whether selection quality matters once recovery is costly: an empty-explorer control in which the explore call returns no files, and a condition that restricts the parent to the files the explorer returned. If the empty explorer matches Jev and search, exploration is not the decision through which Jev can matter for this writer. We will then extend to the remaining 40 sampled Verified tasks (about $9 projected from measured costs). Confidence intervals are not meaningful for five or ten single rollouts (2/5 has a 95% interval of roughly 12% to 77%), so the final study will use repeated rollouts on more issues and paired bootstrap intervals per condition, as in the extended run and the file-selection study. We also plan to move Jev to a decision that the agent cannot bypass: uncertainty-gated escalation between a cheap and a strong writer, with thresholds chosen on independent development data. The original BANKING77/CLINC150 intent-classification track remains deferred. Training with Tinker credits is conditional on a demonstrated need.

Safety considerations. We use public software issues and repository fixtures, not clinical data or deployed systems. Generated patches are research artifacts and are never merged upstream. Credentials are excluded from traces and from agent processes. Agent-side tests run in network-disabled containers on Verified, but the agent's file edits are restricted only by tool permissions on the host, and the development runs execute repository tests on the host without a container.

REFERENCES

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/.

OpenCode contributors. OpenCode 1.19.5, native task/explore implementation. Software source. Accessed November 6, 2026. https://github.com/anomalyco/opencode/blob/v1.19.5/packages/opencode/src/tool/task.ts.

OpenRouter. Jev documentation: using the TypeSafe decision model on OpenRouter. API documentation. Accessed November 6, 2026. https://openrouter.ai/docs/guides/community/jev.

SWE-bench contributors. SWE-bench Lite dataset, development split. Revision 6ec7bb89b9342f664a54a6e0a6ea6501d3437cc2. Accessed November 6, 2026. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Lite.

SWE-bench contributors. SWE-bench Verified dataset, test split. Revision 78f471bf655a3137b2e8a75af1501690ec009ec3. Accessed November 6, 2026. https://huggingface.co/datasets/SWE-bench/SWE-bench_Verified.

AI-USE DISCLOSURE STATEMENT

Rami. OpenAI Codex assisted with interpreting the instructor feedback, inspecting source and documentation, implementing the provider adapter and local evaluator, executing the development pilot, and drafting the first version of this report. **Kristjan.** Anthropic's Claude (Claude Code, Opus 5.5) assisted with Linux reproduction, the API-key loading change, selecting the Verified sample, implementing and checking the Verified runner and analysis, interpreting the results, and revising this report. **Ómar.** Anthropic's Claude Code assisted with reviewing the repository and this report, editing it for the required format and this disclosure, designing, implementing and running the offline file-selection study and shortlist sweep, extending the development runner (oracle condition, timing, additional issues) and running and analyzing the extended development run, and merging the team's report versions. **Within the evaluated system.** Jev 1.14 made file-relevance decisions within the evaluated agent; GPT-6.1-Sol produced the experiments' patches and native exploration responses. Benchmark issues, upstream fixes, and tests come from SWE-bench rather than being generated for this project. The authors reviewed the reported results against the preserved run artifacts and take full responsibility for this report's contents.